The Problem: Failure Propagation in Distributed Systems

Imagine you're on a large ship, cruising across the vast ocean. Suddenly, there's a breach in the hull, and water starts pouring into the ship. What do you think will happen? The ship will start to sink, right? But, does the whole ship sink immediately? Thanks to the architecture of modern ships, the answer is no. The ship is divided into several watertight compartments or 'bulkheads'. If water floods one compartment, the others remain unaffected, at least for some time, buying valuable time for rescue efforts.

But what if our ship didn't have these bulkheads? The water would quickly flood the entire ship, causing it to sink rapidly. The failure (hull breach) would propagate across the entire ship, leading to a total system failure (the ship sinking). This is an example of a cascading failure in a physical system.

Now, let's bring this concept back to distributed systems. Like our ship, a distributed system consists of multiple components (services, processes, etc.). Ideally, these components work together seamlessly to provide a functional system. But, in reality, failures can and do occur.

Cascading Failures in Distributed Systems

A cascading failure in a distributed system is similar to a ship sinking. When one component fails, the failure can propagate to other components, leading to a widespread system failure.

Let's take an example to illustrate this point. Imagine a microservices-based e-commerce platform. You have separate services for user management, inventory management, payment processing, and so on. Now, suppose the inventory service fails due to a database outage. The user service, which relies on the inventory service to display product availability, also starts failing. The payment service, which checks inventory before processing payments, likewise fails.

In a short time, the entire platform becomes unavailable, all because of a failure in one service. This is a classic example of a cascading failure.